Step 1 — A Minimal Server With One Tool — Hints

> Back to this step · Hint 1 · Hint 2 · Solution

Two hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The pieces, and where each one lives
- Hint 2 — The plan, and almost the whole thing

Back to this step · Hint 1 · Hint 2 · Solution

Hint 1 — The pieces, and where each one lives

This step proves one thing only: your code starts an MCP server, and a client (the Inspector or a tiny script) can connect and call its one tool. No database, no second tool yet.

The pieces:

- `src/config.py`, `src/exceptions.py`, `src/utils/logger.py` — Doc01's config/errors/logger, in the `src/` package. The **only** twist: the logger must write to **stderr**, not stdout — on the stdio transport, stdout *is* the protocol channel, so one log line to stdout breaks every message the client reads. `logging.StreamHandler(sys.stderr)` does it.
- `server.py` (at the repo root, imports from `src`):
  - `from mcp.server.mcpserver import MCPServer` — the server class (in the current SDK it's `MCPServer`, not `FastMCP`).
  - `mcp = MCPServer("notes-server")` — build it once.
  - `@mcp.tool()` on `add_note(text: str) -> str` — the decorator turns a plain function into something a client can call. Store the text in a plain Python list.
  - `mcp.run(transport="stdio")` as the last line — nothing listens until this runs.

Don't overthink storage — one in-memory list, appended to. It's *meant* to forget everything on restart; Step 2 fixes that.

Back to this step · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

`server.py` is this shape — fill in the body:

```
# server.py
from mcp.server.mcpserver import MCPServer

from src.config import load_config
from src.utils.logger import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = MCPServer("notes-server")
notes: list = []

@mcp.tool()
def add_note(text: str) -> str:
    """Add a new note to the local notes store. Returns a short
    confirmation once the note is stored."""
    # your turn: append text to notes, log it (logger.info, NEVER print),
    # and return a confirmation mentioning len(notes)
    ...

if __name__ == "__main__":
    logger.info("Starting notes-server over stdio")
    mcp.run(transport="stdio")
```

Then test it. An MCP server can only be driven by a client, so your integration test *is* a tiny client: `stdio_client(StdioServerParameters(command="python", args=["server.py"]))`, open a `ClientSession`, `await session.initialize()`, `await session.list_tools()`, and `await session.call_tool("add_note", {"text": "..."})`. Add `tests/unit/test_config.py` (Doc01's three checks, with a made-up setting name) and `tests/integration/test_server.py` (case 1: the tool is listed and callable). Run both with `python -m`. Compare against the Solution.